Step 2b — Dataset: generare dati con un LLM

Obiettivo

Aggiungere ai dati esistenti esempi che i dataset pubblici non hanno, o hanno in quantità insufficiente:

1. **negativi difficili**: coppie quasi identiche a una positiva, ma non supportate (si toglie l'evidenza dal documento, o si cambia un dettaglio del claim);
2. **FactKG in prosa**: le triple del grafo diventano paragrafi, così il dataset si può usare;
3. **dati multi-hop in stile FactCG (CG2C)** da documenti in prosa;
4. **verifica automatica delle etichette** di tutto ciò che viene generato.

Il prototipo è già nella repo: `llm_gen/` (API NVIDIA, modello DeepSeek-V4.1-Flash), con i task `remove_evidence`, `contradict_claim`, `verbalize_kg`, `verify_label`.

Regola preliminare: si genera solo dal train

Tutti i task di questo step partono **solo da righe dello split di train** congelato allo Step 0. Se si partisse da righe di dev o test, documenti di valutazione finirebbero nel training in forma modificata (es. con l'evidenza cancellata). Le coppie generate ereditano il `doc_id` della riga di partenza, quindi restano nel train. In dev e test restano solo etichette originali.

Con il prototipo: `llm_gen/run.py --source data/finetune/splits/train/<fonte>.jsonl` (default `--split train`).

Principio: l'LLM modifica, la struttura etichetta

Un LLM che decide da solo se un claim è supportato introduce i suoi errori nelle etichette. Per questo ogni task è costruito in modo che **l'etichetta derivi dall'operazione**, non dal giudizio dell'LLM:

Operazione	Etichetta che ne deriva	Perché è affidabile
Cancellare dal documento la frase che stabilisce una parte del claim	0 (non stabilito)	se l'evidenza non c'è più, il claim non può essere supportato
Cambiare un'entità o un numero del claim in un valore che il documento esclude	0 (contraddetto)	il documento dice il contrario
Riscrivere in prosa le triple di un claim FactKG	stessa etichetta di FactKG	il contenuto informativo non cambia (se la riscrittura è fedele)
Scrivere un claim che esprime un sottografo estratto dal documento (CG2C)	1	ogni fatto del claim viene da un arco del documento

Poi un **secondo passaggio** (un LLM giudice, anche diverso dal primo) controlla il risultato, e si tengono solo gli esempi su cui operazione e giudice concordano.

1. Negativi per cancellazione dell'evidenza (`remove_evidence`)

Da supportato a non supportato, senza contraddizione. È il caso più importante per un fact checker: insegna che "il documento non lo dice" vale "no", anche quando il claim è plausibile.

Esempio (MiniCheck, riga reale):

> Documento originale: "... an unprecedented number of participants shattered the once-daunting 4-minute mile barrier. **The official tally confirmed the breakthrough, with exactly 13 runners registering times below the previous record.** This feat not only rewrote the record books ..."
>
> Claim: "Thirteen men finished the mile faster than 4:00:00." — label 1
>
> Documento modificato (cancellata la frase in grassetto) → stessa coppia, **label 0**: il documento dice che "un numero senza precedenti" di corridori ha battuto la barriera, ma non dice quanti.

Prompt (in `llm_gen/tasks.py`): l'LLM deve **solo cancellare**, il minimo indispensabile, senza far nascere una contraddizione, e restituire il documento modificato e le parti tolte in JSON.

Controlli automatici già implementati: documento più corto; quasi tutte le parole rimaste erano già nel documento (cioè cancellazioni, non riscritture); elenco delle parti tolte non vuoto.

2. Negativi per modifica minima del claim (`contradict_claim`)

Da supportato a contraddetto. Si cambia una sola cosa del claim.

> Documento: "The meeting was moved to Tuesday at 10:00. It will be held in Rome." Claim originale (1): "The meeting is on Tuesday in Rome." Claim generato (0): "The meeting is on **Wednesday** in Rome."

Regole nel prompt: un solo cambiamento, il resto parola per parola; la contraddizione deve risultare dal documento, non da conoscenze esterne; niente "not" aggiunto se esiste un cambiamento migliore (le negazioni sono una scorciatoia nota di FEVER).

Controlli automatici: claim cambiato di poco; la frase di evidenza citata dall'LLM compare davvero nel documento; segnalazione se l'LLM ha solo aggiunto una negazione.

3. FactKG in prosa (`verbalize_kg`)

Perché. Il "documento" di FactKG costruito oggi è una lista di triple `soggetto | relazione | oggetto`. In uso il modello leggerà prosa: le triple vanno riscritte.

Come.

1. Si passano all'LLM **solo le triple**, mai il claim (altrimenti il paragrafo ne copierebbe la formulazione e il compito diventerebbe banale).
2. Si limitano a ~25 triple, tenendo prima quelle delle entità più ricorrenti (da cui partono i cammini di evidenza): i documenti di FactKG ne hanno a volte più di 50, molte rumorose.
3. L'LLM scrive un paragrafo enciclopedico con tutti e soli quei fatti.
4. Controllo: `verify_label` sulla coppia (paragrafo, claim) deve dare lo stesso verdetto di FactKG; altrimenti si scarta (il paragrafo ha perso un fatto, o il taglio a 25 triple ha tolto un'evidenza).

Esempio (riga reale, triple selezionate):

> Triple: `Anders Osborne | record label | Okeh Records`, `Okeh Records | parent company | Sony Music Entertainment`, `Anders Osborne | genre | Blues`, `Anders Osborne | birth date | 1966-05-04`, …
>
> Paragrafo atteso: "Anders Osborne, born on 4 May 1966, is a blues and rock guitarist and singer who has recorded for labels including Okeh Records, Alligator Records and Shanachie Records. Okeh Records is part of Sony Music Entertainment …"

4. Dati multi-hop in stile FactCG (CG2C)

Cosa sono. Il metodo CG2C di FactCG (paper) genera claim multi-hop da documenti in prosa: dal documento si estrae un grafo (entità e relazioni, ciascuna legata alla sua frase), si sceglie un cammino di 2–4 archi, un LLM scrive un claim che lo esprime (positivo); per i negativi si sostituisce un'entità del cammino o si cancella la frase di un arco. Con questi dati un modello da 0,4B arriva a 75,6 su LLM-AggreFact.

Disponibilità. Il repository di FactCG (MIT) rilascia codice e modello, non i dati: vanno rigenerati.

Come da noi. Documenti di partenza in prosa e non presenti nei benchmark: paragrafi HoVer (`Datasets/HoVer/data/wiki_wo_li`
pagine FEVER, notizie recenti, trascrizioni di meeting decontaminate. Un task `cg2c` andrà aggiunto a `llm_gen/tasks.py` con tre chiamate per documento (estrazione del grafo, claim positivo, claim negativo) e verifica finale.

Esempio:

> Documento: "Nancy R. Howell is Professor of Theology at Saint Paul School of Theology in Kansas City, Missouri. […] Saint Paul School of Theology is a seminary affiliated with the United Methodist Church."
>
> Cammino: Howell —lavora a→ Saint Paul School of Theology —affiliata a→ United Methodist Church

Claim positivo: "Nancy R. Howell teaches at a seminary affiliated with the United Methodist Church." (1)

Claim negativo (entità sostituita): "Nancy R. Howell teaches at a seminary affiliated with the **Presbyterian** Church." (0)

5. Verifica delle etichette (`verify_label`)

Ogni esempio generato passa da un giudice LLM che risponde *supported* / *contradicted* / *not_enough_info*. Si tengono solo gli esempi in cui:

- per `remove_evidence`: il giudice dice *not_enough_info* (non *contradicted*: vorrebbe dire che la cancellazione ha creato una contraddizione);
- per `contradict_claim`: il giudice dice *contradicted*;
- per `verbalize_kg` e CG2C: il giudice concorda con l'etichetta.

Per ridurre gli errori correlati, conviene usare come giudice un modello **diverso** dal generatore (es. genera Qwen3-30B-A3B in locale, giudica gpt-oss-120b o un'API). Tasso di scarto atteso: 10–30%. Se supera il 40%, il prompt di generazione va rivisto.

6. Quale LLM usare

Uso	Locale (L40S)	API gratuita	A pagamento
Prove su pochi esempi	—	NVIDIA NIM (DeepSeek-V4.1-Flash, ~1.000 crediti, 40 richieste/min)	—
Generazione in volume	Qwen3-30B-A3B-Instruct (FP8, ~31 GB), gpt-oss-20b	Mistral "Experiment" (~1 miliardo di token/mese)	Gemini Flash, Claude Haiku 4.5, DeepSeek (con API batch, ~50% di sconto)
Giudice / verifica	Gemma-4-31B-it, gpt-oss-120b (2× L40S)	Gemini Flash	Claude Sonnet 5.5 / Opus 5.5, GPT-5.x

Attenzione alle condizioni d'uso dei fornitori commerciali: alcune vietano di usare gli output per addestrare modelli concorrenti. Con i modelli aperti in locale il problema non c'è.

7. Volumi consigliati

Tipo	Quantità per il pilota	Quantità per la corsa piena
`remove_evidence` (da MiniCheck, HoVer, FEVER, SummEdits)	2.000	6.000–10.000
`contradict_claim`	2.000	6.000–10.000
FactKG in prosa	0 (solo esperimento D3)	5.000–15.000 se l'esperimento è positivo
CG2C	0 (esperimento D9)	5.000–10.000

Costo indicativo: ~1.500 token per chiamata (documento + risposta), più una chiamata di verifica. 4.000 esempi verificati ≈ 8.000 chiamate ≈ 12M token: poche ore con un modello locale.

8. Comandi (prototipo esistente)

```
python llm_gen/run.py --task remove_evidence --source minicheck --n 2 --dry-run   # solo i prompt
python llm_gen/run.py --task remove_evidence --source minicheck --n 2
python llm_gen/run.py --task contradict_claim --source hover --n 2
python llm_gen/run.py --task verbalize_kg --source factkg --n 2
python llm_gen/run.py --task verify_label --source llm_gen/outputs/<file del run precedente>.jsonl
↪  --n 2
```

Ogni coppia generata ha lo stesso formato di `build_pairs.py`, lo stesso `doc_id` della coppia di partenza (così resta nello stesso split) e `meta.pair_id` che collega le due coppie (utile per la loss contrastiva, 04_loss.md).

Da aggiungere per l'uso in volume: un endpoint locale (vLLM, compatibile OpenAI) nel client, esecuzione parallela, ripresa dopo interruzioni, e uno script che raccoglie le `new_row` verificate in `data/finetune/raw/<fonte>+<task>.jsonl`.

Criteri per passare allo step successivo

- Per ogni task: tasso di accettazione (controlli automatici + giudice) misurato su almeno 200 esempi.
- Campione di 30 esempi accettati letto a mano: meno di 3 con etichetta sbagliata.
- File delle coppie verificate pronti per il mix (02e_mix_finale.md).